Daily Quant Digest: 2026-11-08

Foundations Restart, Revision Day 2 (Sunday). No new lessons today. This week we climbed from Lesson 7 to Lesson 11 or 12 in each track. Today we look back down the staircase: one picture per lesson, one older lesson brought back, then fresh problems that make you use several lessons at once.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Revision: Lessons 7 to 11 (independence, total probability, Bayes, classic puzzles, random variables); interview question: two coins in a bag	Easy-Medium
Derivatives	FR Revision: Lessons 7 to 12 (futures, bonds, duration, yield curve, repo, swaps) on one repo-financed bond and its swap hedge	Easy-Medium
Algorithms & DS	FR Revision: Lessons 7 to 12; variant problem: shortest run of days whose volume reaches a target (sliding window vs binary search on the answer)	Medium
Music	Derek and the Dominos, "Layla" (1970)	

1. Probability & Statistics: The Week on One Page

> **Where we are.** Lessons 1 to 5 built the **room of possible worlds** and taught us to count the worlds. Lesson 6 showed that news **moves the walls** in to a smaller room. This week used that one move in five different ways, and then, in Lesson 11, started putting a number on every world.

1.1 The week in five pictures

Lesson	The picture to remember	The rule it gives
L7: independence	**The walls move, the share stays.** News about B does not change the share of the room where A happens	$P(A \cap B) = P(A)\,P(B)$, same as $P(A \mid B) = P(A)$
L8: total probability	**Water through pipes.** Split the flow into cases, multiply along each pipe, add the pipes	$P(A) = \sum_i P(A \mid B_i)\,P(B_i)$
L9: Bayes' rule	**The airport beep.** Look only at the people who beeped and ask who they are	$P(B \mid A) = \dfrac{P(A \mid B)\,P(B)}{P(A)}$; posterior odds = prior odds x likelihood ratio
L10: classic puzzles	**The rule that produced the news matters** (Monty Hall, boy-or-girl); **count handshakes** (birthdays)	Ask "how did I get told this?"; n people make $\binom{n}{2}$ pairs
L11: random variables	**A price tag on every world**; the PMF sorts worlds into buckets by tag; the CDF is a tank filling up	$p(x) = P(X = x)$; $F(x) = P(X \le x)$; $P(a < X \le b) = F(b) - F(a)$

Read the table as one story. L6 gave us the smaller room. L7 asks: did the smaller room change anything? L8 builds a probability from smaller rooms. L9 runs L8 backwards: from the evidence back to the cause. L10 warns that the exact wording of the news decides the room. L11 then stops asking "did it happen?" and starts asking "how much?".

1.2 How L8 and L9 fit together: the same tree, read two ways

This is the most useful idea of the week, so here it is once more in one picture.

Draw a tree. The first split is the **cause** (which desk, which coin, which kind of day). The second split is the **evidence** (alarm or no alarm, heads or tails).

- **Lesson 8 reads the tree forwards.** "How likely is the evidence?" Add up every path that ends in that evidence.
- **Lesson 9 reads the tree backwards.** "The evidence happened. Which cause was it?" Take the paths that end in the evidence, and ask what share of them started at each cause.

So Bayes' rule is just "one path divided by all the paths that end in the same place". The denominator of Bayes **is** the law of total probability. If you remember that, you never need to memorise the formula.

1.3 One older lesson revisited: the addition rule (L2) meets independence (L7)

Two weeks ago, Lesson 2 gave us the addition rule: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. The minus sign removes the people we counted twice. The hard part was always the last term: how big is the overlap?

Independence answers that question. If A and B are independent, the overlap is just $P(A)\,P(B)$.

A concrete scene. A trading desk has two separate risk systems. System 1 goes down on 1% of days. System 2 goes down on 2% of days. They run on separate hardware, so assume they fail independently. On what share of days is at least one system down?

Imagine 10,000 days. System 1 is down on 100 of them. System 2 is down on 200. Both are down on $1\% \times 2\% = 0.02\%$ of days, which is 2 days. Those 2 days were counted in both groups, so:

$$100 + 200 - 2 = 298 \text{ days, so } P(\text{at least one down}) = 2.98\%.$$

The faster road: count the opposite. "At least one down" is the opposite of "both up". Both up happens with probability $0.99 \times 0.98 = 0.9702$, so the answer is $1 - 0.9702 = 0.0298$. Same number.

Where the shortcut breaks. If both systems share one power supply (a **common cause**, from Lesson 7), they are not independent. The overlap is bigger than 2 days, and "at least one down" is smaller than 2.98%, but "both down together" becomes much more likely. Diversification only works when failures are really independent.

1.4 Practice problems (answers in 1.7)

1. **(L7)** Roll two fair dice. Let A = "the sum is even" and B = "the first die shows 3". Are A and B independent? Now let C = "the product is even" and D = "the first die is even". Are C and D independent?
2. **(L8)** On an FX desk, 60% of days are calm and 40% are volatile. A stop-loss order gets hit on 5% of calm days and on 20% of volatile days. On what share of days is the stop hit?
3. **(L9)** The stop was hit today. What is the probability that today was a volatile day?
4. **(L11)** Roll two fair dice and let M be the larger of the two numbers. Find the PMF of M and $P(M \geq 5)$.
5. **(L10)** Five traders each pick a random day of the week (all 7 days equally likely, independently) for their day off. What is the probability that at least two of them pick the same day?

1.5 Interview question: two coins in a bag

(A warm-up of the kind Jane Street and SIG like. It uses total probability, Bayes, independence and a PMF in one go. The trap is part (c).)

A bag has two coins. One is fair. The other is bent and lands heads with probability $3/4$. You pick a coin at random (each with probability $1/2$) and flip **the same coin** twice.

a. What is the probability that the first flip is heads?

b. What is the probability that both flips are heads?

c. Are the two flips independent?

d. Given that the first flip is heads, what is the probability that the second flip is heads?

e. Let X be the number of heads in the two flips. Write down the PMF and the CDF of X.

1.6 Solution

Step 0: count people first. Imagine 3,200 people each play this game. Half of them, 1,600, get the fair coin and 1,600 get the bent coin.

- **Fair coin, 1,600 people.** Each of HH, HT, TH, TT happens to a quarter of them: 400 each.
- **Bent coin, 1,600 people.** HH: $1{,}600 \times \frac{3}{4} \times \frac{3}{4} = 900$. HT: $1{,}600 \times \frac{3}{4} \times \frac{1}{4} = 300$. TH: 300. TT: $1{,}600 \times \frac{1}{4} \times \frac{1}{4} = 100$.

	HH	HT	TH	TT	Total
Fair coin	400	400	400	400	1,600
Bent coin	900	300	300	100	1,600
Total	1,300	700	700	500	3,200

Every answer below is read off this table.

(a) First flip heads (L8, water through pipes). People whose first flip is heads: the HH and HT columns, $1{,}300 + 700 = 2{,}000$.

$$P(H_1) = \frac{2{,}000}{3{,}200} = \frac{5}{8}.$$

With the formula: $P(H_1) = P(H_1 \mid \text{fair})\,P(\text{fair}) + P(H_1 \mid \text{bent})\,P(\text{bent}) = \frac{1}{2}\cdot\frac{1}{2} + \frac{3}{4}\cdot\frac{1}{2} = \frac{1}{4} + \frac{3}{8} = \frac{5}{8}$.

(b) Both heads. The HH column: $1{,}300/3{,}200 = 13/32 \approx 0.406$.

(c) Independent? No. If the flips were independent, we would have $P(HH) = P(H_1)\,P(H_2) = \frac{5}{8} \times \frac{5}{8} = \frac{25}{64}$. But $P(HH) = \frac{13}{32} = \frac{26}{64}$. Close, but not equal, so the flips are **not** independent.

Why? Once you know which coin you hold, the flips are independent: a coin has no memory. But you do **not** know which coin you hold, and the first flip leaks information about it. A head makes the bent coin more likely, and that makes another head more likely.

This is the **common cause** from Lesson 7: the unknown coin sits behind both flips, just like the shared power supply sat behind both risk systems.

(d) Second head given the first (L6 and L9). The walls move in to the 2,000 people whose first flip was heads. Of those, 1,300 also got a second head:

$$P(H_2 \mid H_1) = \frac{1{,}300}{2{,}000} = \frac{13}{20} = 0.65.$$

That is bigger than $5/8 = 0.625$: the first head pushed our belief towards the bent coin.

The Bayes road to the same number. After one head, which coin do we hold? Of the 2,000 people with a first head, $400 + 400 = 800$ hold the fair coin and $900 + 300 = 1{,}200$ hold the bent coin. So $P(\text{bent} \mid H_1) = 1{,}200/2{,}000 = 3/5$. In odds form: prior odds bent to fair are 1 to 1, the likelihood ratio of a head is $\frac{3/4}{1/2} = 1.5$, so posterior odds are 1.5 to 1, which is $3/5$. Now flip again with the updated belief:

$$P(H_2 \mid H_1) = \frac{3}{5} \cdot \frac{3}{4} + \frac{2}{5} \cdot \frac{1}{2} = \frac{9}{20} + \frac{4}{20} = \frac{13}{20}.$$

(e) PMF and CDF (L11). Sort the 3,200 people into buckets by their tag X = number of heads. Zero heads: TT, 500 people. One head: HT and TH, 1,400. Two heads: HH, 1,300.

x	**0**	**1**	**2**
PMF $P(X = x)$	$5/32$	$14/32$	$13/32$
CDF $P(X \le x)$	$5/32$	$19/32$	1

Interview trick. The PMF of a **mixture** is the average of the two PMFs. Fair coin: $(\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$. Bent coin: $(\frac{1}{16}, \frac{6}{16}, \frac{9}{16})$. Average them: $(\frac{5}{32}, \frac{14}{32}, \frac{13}{32})$. That is just Lesson 8 applied to every bucket at once.

Sanity checks.

- The PMF adds up to $(5 + 14 + 13)/32 = 1$. Good.
- Every answer sits between the fair-coin answer and the bent-coin answer: $P(H_1) = 0.625$ is between 0.5 and 0.75; $P(HH) = 0.406$ is between 0.25 and 0.5625. A mixture always lands in between.
- $P(H_2 \mid H_1) = 0.65$ is bigger than $P(H_2) = 0.625$. Heads "attract" heads, as the common cause story predicts.

1.7 Answers to the practice problems

1. **A and B: independent. C and D: not.** $P(A) = 18/36 = 1/2$. Given the first die is 3, the sum is even only if the second die is odd: $3/6 = 1/2$. The share stayed the same, so independent. For C: the product is odd only if both dice are odd, $9/36$, so $P(C) = 27/36 = 3/4$. Given the first die is even, the product is **always** even, so $P(C \mid D) = 1 \neq 3/4$. Not independent.

2. **11%.** Pipes: $0.6 \times 0.05 + 0.4 \times 0.20 = 0.03 + 0.08 = 0.11$. With 1,000 days: 30 calm days with a hit plus 80 volatile days with a hit, 110 in total.
3. **8/11, about 72.7%.** Look only at the 110 days with a hit; 80 of them were volatile. Notice how the news moved us: volatile days were 40% of all days but 73% of the days with a hit.
4. $P(M = k) = (2k - 1)/36$**, and** $P(M \geq 5) = 5/9$**.** Use the CDF first: $M \leq k$ means both dice are at most k, which happens in $k \times k$ of the 36 worlds, so $F(k) = k^2/36$. The PMF is the step in the tank: $P(M = k) = \frac{k^2 - (k-1)^2}{36} = \frac{2k-1}{36}$, giving $\frac{1}{36}, \frac{3}{36}, \frac{5}{36}, \frac{7}{36}, \frac{9}{36}, \frac{11}{36}$. Then $P(M \geq 5) = 1 - F(4) = 1 - 16/36 = 20/36 = 5/9$.
5. **About 85.0%.** Count the opposite, all different, by walking in one at a time: $\frac{7}{7} \cdot \frac{6}{7} \cdot \frac{5}{7} \cdot \frac{4}{7} \cdot \frac{3}{7} = \frac{2{,}520}{16{,}807} \approx 0.150$. So $P(\text{some match}) \approx 1 - 0.150 = 0.850$. Five people make $\binom{5}{2} = 10$ handshakes, each a 1 in 7 chance of matching, which is why a match is so likely.

---

2. Derivatives: One Bond, Six Lessons

> **Where we are.** Lessons 1 to 6 gave us interest, discounting, how a market quotes, FX, and forwards. This week, Lesson 7 turned forwards into **futures** with daily margin. Lessons 8 to 12 moved into **rates**: bonds, duration, the yield curve, repo and swaps. Today we follow **one trade** from start to finish and use every one of those lessons.

2.1 The week in six pictures

Lesson	The picture to remember	The rule it gives
L7: futures	**A bar tab vs paying per round**: futures settle gains and losses every day	Daily variation margin; same total P&L as a forward, different timing
L8: bonds	**A flat with a fixed rent**: the rent (coupon) is fixed, so the price moves	Price = PV of all payments; yield up means price down
L9: duration and DV01	**Sandbags on a plank**: duration is the balance point of the payments	$DV01 \approx D_{\text{mod}} \times \text{value} \times 0.0001$
L10: yield curve	**A menu of lease prices** for different lengths	Forward rates by “two roads to the same place”
L11: repo	**A pawn shop for bonds**: leave the bond, take cash, buy it back	Interest $= \text{cash} \times r \times \text{days}/360$; haircut = your own money
L12: swaps	**Swapping electricity contracts**: fixed price for floating price	Par rate $= \dfrac{1 - DF_n}{\sum_i DF_i}$; DV01 $\approx N \times \sum_i DF_i \times 0.0001$

2.2 The scene

A Hong Kong bank's treasury desk thinks a 4-year USD bond is cheap. It wants to **earn the bond's coupon** without tying up much cash, and it does not want to bet on the direction of interest rates. Its plan has three parts:

1. **Buy** USD 50 million face value of the bond.
2. **Finance** it with repo: pawn the bond for cash.
3. **Hedge** the rate risk with an interest rate swap.

To keep the arithmetic easy, the bond pays its coupon **once a year** (real Treasuries pay twice a year). The market data:

Market data	Value
Bond	4 years, 4.00% annual coupon, yield 4.20%
Zero rates (annual compounding) for 1, 2, 3, 4 years	3.80%, 3.90%, 4.00%, 4.10%
30-day repo rate	3.95% (ACT/360), haircut 2%

Try each step yourself before reading the answer.

2.3 Step 1 (L8): what does the bond cost?

The bond pays 4 per 100 face each year, and 104 at the end. Its price is the present value of those payments, discounted at its yield of 4.20%:

$$P = \frac{4}{1.042} + \frac{4}{1.042^2} + \frac{4}{1.042^3} + \frac{104}{1.042^4} = 3.839 + 3.684 + 3.536 + 88.219 = 99.277.$$

In words: each payment is shrunk by one more factor of 1.042 for each year we wait, then we add them up. The price is a little **below** 100 because the coupon (4.00%) is below the yield (4.20%): the rent is lower than the market rent, so the flat sells at a discount.

For USD 50 million face, the bond costs $50{,}000{,}000 \times 0.99277$ = **USD 49,638,715**.

2.4 Step 2 (L9): how much does it lose if rates rise?

Weight each payment's time by its share of the price, and you get the **Macaulay duration**, the balance point of the sandbags: $D = 3.774$ years. Divide by $1 + y$ to get the **modified duration**: $D_{\text{mod}} = 3.774/1.042 = 3.622$.

$$DV01 = D_{\text{mod}} \times \text{value} \times 0.0001 = 3.622 \times 49{,}638{,}715 \times 0.0001 \approx \text{USD } 17{,}979.$$

In words: for every 1 basis point rise in the yield, the position loses about USD 18,000. A 10 bp rise costs about USD 180,000. (Bumping the yield by 1 bp up and down and repricing gives the same 17,979, which is a good check.)

2.5 Step 3 (L11): financing with repo

The desk pawns the bond. With a 2% haircut, the repo lender gives 98% of the bond's value:

Item	Amount (USD)
Bond value	49,638,715
Cash borrowed in repo (98%)	48,645,941
Desk's own money (the 2% haircut)	992,774

Cost of 30 days of repo: $48{,}645{,}941 \times 3.95\% \times 30/360$ = **USD 160,126**.

Coupon earned over 30 days: $50{,}000{,}000 \times 4.00\% \times 30/360$ = **USD 166,667**.

So the **carry** is about $166{,}667 - 160{,}126$ = **+USD 6,540** a month (a little more in reality, because a discount bond also drifts up towards 100 over time, the pull to par from Lesson 8).

Now the danger. The desk put in only USD 0.99 million but carries the full DV01 of USD 17,979. That is **50 times leverage** $(1/2\%)$. A rise in yield of just

$$\frac{992{,}774}{17{,}979} \approx 55 \text{ bp}$$

wipes out the desk's money completely. A small carry of USD 6,540 a month against a possible loss of USD 1 million is not a good deal on its own. It needs a hedge.

2.6 Step 4 (L10 and L2): reading the curve

First the discount factors, from the zero rates (Lesson 2: future money is a foreign currency, and DF is its exchange rate into today's money):

Year	Zero rate	$DF = 1/(1+z)^t$	Forward rate for that year
1	3.80%	0.96339	3.80%
2	3.90%	0.92634	4.00%
3	4.00%	0.88900	4.20%
4	4.10%	0.85152	4.40%

The forward rate by two roads (L10). Road 1: lend for 2 years at 3.90%. Road 2: lend for 1 year at 3.80%, then roll for one more year at the forward rate f. Both roads must end in the same place:

$$1.039^2 = 1.038 \times (1+f), \quad \text{so} \quad 1+f = \frac{1.079521}{1.038} = 1.04000, \quad f = 4.00\%.$$

The curve slopes up, so the forward rates climb even faster than the zero rates: 3.80%, 4.00%, 4.20%, 4.40%.

2.7 Step 5 (L12): the swap hedge

The desk enters a 4-year swap where it **pays fixed** and receives floating (SOFR). Paying fixed gains when rates rise, which is exactly the opposite of the bond.

The fair fixed rate. The sum of the discount factors (the **annuity**) is

$$A = 0.96339 + 0.92634 + 0.88900 + 0.85152 = 3.63025.$$

$$\text{Par swap rate} = \frac{1 - DF_4}{A} = \frac{1 - 0.85152}{3.63025} = 4.090\%.$$

Check: it should be a DF-weighted average of the forwards 3.80% to 4.40%, and 4.09% sits right in that range, a little below the middle because the early years get bigger weights.

How big a swap? A swap's DV01 is about $N \times A \times 0.0001$. To match the bond's DV01:

$$N = \frac{17{,}979}{3.63025 \times 0.0001} \approx \text{USD } 49.5 \text{ million}.$$

2.8 Step 6 (L7): the swap settles like a futures contract

The swap is **cleared** through a central counterparty, just like the futures in Lesson 7. That means a bar tab that is settled every day.

Scene: rates jump 10 bp overnight. The bond loses about USD 180,000. The swap gains about USD 180,000, and the clearing house pays that gain to the desk **in cash** the next morning as variation margin. At the same time, the repo lender sees the bond worth less and asks for more cash to restore the 2% haircut, about USD 176,000 (98% of the loss). The swap cash pays for it. The hedge protects both the **P&L** and the **cash**.

What the hedged trade is really betting on. With rates hedged, the desk earns the bond's yield (4.20%) and pays the swap rate (4.09%): about **+11 bp a year**, plus any gap between SOFR (received on the swap) and the repo rate (paid on the loan). Small, so desks do it in size, and that is where the risk comes from.

When it went wrong. In March 2020, many funds held Treasuries financed in repo and hedged. When markets panicked, haircuts and margin calls jumped at the same moment, and the hedges did not always pay cash where and when it was needed. Forced sellers dumped Treasuries, and the Federal Reserve had to step in and buy them. The lesson: a hedge can make P&L flat and still leave you short of cash.

2.9 Likely interview questions

1. **"Your bond is hedged with a swap. How can you still lose money?"** If the bond yield and the swap rate move by different amounts (the spread changes), if the curve twists (the DV01s match only for parallel moves), or if funding gets more expensive (repo rate up, haircut up). The last one can force you to sell at the worst time.

2. **“Why is the swap DV01 per unit of notional so close to the bond’s?”** A par swap behaves like a par bond financed at the floating rate. Per 100, the bond’s DV01 is $3.622 \times 99.28 \times 0.0001 = 0.0360$, and the swap’s is $3.630 \times 0.0001 = 0.0363$. Almost the same, which is why the swap notional (49.5m) is close to the bond face (50m).
3. **“Why does a 2% haircut matter so much?”** It sets the leverage at 50 times. With a DV01 of about USD 18,000 per basis point, a 55 bp move uses up the whole USD 1 million of the desk’s own money. Halve the haircut and the move that wipes you out halves too.

---

3. Algorithms & Data Structures: The Train Window and the Dial

> **Where we are.** Lessons 1 to 6: Big-O, C++ basics, scanning arrays, hash maps, prefix sums and sorting. This week: two pointers (L7), sliding window (L8), recursion (L9), binary search (L10), binary search on the answer (L11) and stacks (L12). Today’s problem can be solved with **two** of this week’s ideas, and comparing them teaches more than either one alone.

3.1 The week in six pictures

Lesson	The picture to remember	The rule it gives
L7: two pointers	**Two helpers at both ends of a sorted shelf**	Each step throws away a whole row of pairs safely: $O(n)$
L8: sliding window	**A train window**: grow at the front, repair at the back, record	Each index enters once and leaves once: amortised $O(n)$
L9: recursion	**A chain of helpers with sticky notes**	Always a base case; each call is a frame on the call stack
L10: binary search	**A red/green fence**: find the first green post	Half-open $[lo, hi)$; $mid = lo + (hi - lo)/2$; $O(\log n)$
L11: binary search on the answer	**A dial with a yes/no checker**	If “works for x” means “works for every bigger x”, search the dial
L12: stacks	**A pile of paused jobs**	Last in, first out; `push`, `top`, `pop` in $O(1)$

3.2 The problem

An execution desk wants to buy a big block of shares. It needs a stretch of **consecutive** trading days whose total market volume is at least T, so the order can hide inside normal trading. Given the daily volumes, find the **length of the shortest** such stretch. If even all the days together fall short, return 0.

Input. An array `vol` of n positive integers, and a target T.

Constraints. $1 \le n \le 10^5$, $1 \le vol[i] \le 10^4$, $1 \le T \le 10^9$.

Examples.

`vol`	T	Answer	Why
`[2, 3, 1, 2, 4, 3]`	7	2	Days `[4, 3]` sum to 7
`[1, 4, 4]`	4	1	One day of 4 is enough
`[1, 1, 1, 1, 1, 1, 1, 1]`	11	0	All days together give only 8

3.3 Brute force first

Try every starting day i. Walk forward, adding volumes, until the total reaches T. Then stop: going further from the same start can only make the stretch longer. Keep the shortest length found.

In the worst case (when T is large) each start walks almost to the end, so this is about $n^2/2$ steps. For $n = 10^5$ that is 5×10^9 steps: far too slow. But it is a perfect **referee** for testing, so the code keeps it.

3.4 Insight 1: the train window (L8)

Picture the days as a long train of carriages and a window that can stretch and shrink.

- **Grow:** move the front edge forward, adding one more day's volume.
- **The window is good** (total at least T): record its length, then **shrink** from the back to see whether a shorter window still works.
- Keep shrinking while the window is still good.

Why is it safe to shrink and never look back? Because all volumes are **positive**. Suppose the window from `left` to `right` is good. Any window that starts at `left` and ends later than `right` is longer, so it cannot beat what we just recorded. So we are done with `left` forever and can move it on.

Walkthrough on `[2, 3, 1, 2, 4, 3]`, $T = 7$:

`right` adds	Window	Sum	Action
2	[2]	2	grow
3	[2,3]	5	grow
1	[2,3,1]	6	grow
2	[2,3,1,2]	8	good: best = 4; drop 2, sum 6, not good
4	[3,1,2,4]	10	good: best = 4; drop 3, sum 7: [1,2,4] good, best = 3; drop 1, sum 6
3	[2,4,3]	9	good: best = 3; drop 2, sum 7: [4,3] good, best = **2**; drop 4, sum 3

Each day is added once and dropped at most once, so there are at most $2n$ moves: $O(n)$ time and $O(1)$ extra space.

3.5 Insight 2: the dial (L11, with L5 prefix sums)

Turn the question around. Instead of "what is the shortest length?", ask a yes/no question about a dial set to length L:

> "Is there **some** window of exactly L days with total at least T?"

Is the answer monotone? Yes. If a window of length L works, stretch it by one day and the total only grows (volumes are positive), so length $L+1$ works too. The fence looks like red, red, red, green, green, green. Binary search finds the first green post.

To answer the yes/no question fast, use **prefix sums** from Lesson 5 (an older lesson worth revisiting): with $P_0 = 0$ and $P_{i+1} = P_i + vol[i]$, the sum of the L days starting at s is $P_{s+L} - P_s$. One check scans all starts in $O(n)$.

On the example: the dial runs from 1 to 6. Check $L = 3$: windows sum to 6, 6, 7, 9. Yes, so the answer is at most 3. Check $L = 2$: sums 5, 4, 3, 6, 7. Yes, at most 2. Check $L = 1$: biggest day is 4. No. Answer: **2**.

Cost: $O(\log n)$ checks of $O(n)$ each, so $O(n \log n)$ time and $O(n)$ space for the prefix array.

Which one to use? The window is faster and uses less memory. The dial is still worth knowing: it works for more questions ("is there a window of length L whose **average** is at least x?") where the window trick has no simple rule for when to shrink.

Where both break. If volumes could be **negative** (say, daily P&L instead of volume), shrinking a good window could make it better, and stretching it could make it worse. Both tricks fail. That version needs a **monotonic deque**, which is coming in Lessons 13 and 14.

3.6 C++ solution

```cpp
#include <algorithm>
#include <vector>
using namespace std;

// Sliding window: O(n) time, O(1) extra space.
int shortestRunWindow(const vector<int>& vol, long long target) {
    const int n = static_cast<int>(vol.size());
    int best = n + 1;          // "infinity": longer than any real window
    long long sum = 0;         // sum of vol[left .. right]
    int left = 0;
    for (int right = 0; right < n; ++right) {
        sum += vol[right];                    // grow
        while (sum >= target) {               // window is good
            best = min(best, right - left + 1);// record
            sum -= vol[left];                 // shrink
            ++left;
        }
    }
    return best == n + 1 ? 0 : best;
}

// Yes/no checker for the dial: does some window of length len reach target?
bool someWindowReaches(const vector<long long>& prefix, int len, long long target) {
    const int n = static_cast<int>(prefix.size()) - 1;
    for (int start = 0; start + len <= n; ++start) {
        if (prefix[start + len] - prefix[start] >= target) return true;
    }
    return false;
}

// Binary search on the answer: O(n log n) time, O(n) space.
int shortestRunBinarySearch(const vector<int>& vol, long long target) {
    const int n = static_cast<int>(vol.size());
    vector<long long> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + vol[i];

    if (prefix[n] < target) return 0;          // even all days fall short

    int lo = 1, hi = n;                        // the answer is in [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (someWindowReaches(prefix, mid, target)) hi = mid;  // green: try shorter
        else lo = mid + 1;                                     // red: go longer
    }
    return lo;
}
```

The full file `2026-11-08.cpp` also has the brute-force referee, the examples, a test of 20,000 random cases checking all three functions agree, and a 100,000-day test whose total is exactly 10^9.

3.7 Walkthrough of the C++

- **`const vector<int>& vol`.** A reference to a constant vector (Lesson 2: a reference is a nickname, not a copy). We read 100,000 numbers without copying them, and `const` promises we will not change them.
- **`static_cast<int>(vol.size())`.** `size()` returns an **unsigned** type (`size_t`). Mixing unsigned and signed numbers is a classic trap: `0u - 1` is a huge positive number, not -1. Casting once to `int` at the top keeps all the index arithmetic signed and safe.
- **`long long sum`.** The total can reach $10^5 \times 10^4 = 10^9$, which fits in a 32-bit `int` (max about 2.1×10^9), but only just. Any bigger input would overflow silently. `long long` (64-bit, max about 9.2×10^{18}) removes the worry. The target is also `long long`, so the comparison `sum >= target` happens between two 64-bit numbers.
- **`int best = n + 1;`** A "sentinel" value meaning "nothing found yet". No real window can be longer than n, so if `best` is still $n + 1$ at the end, no window worked. The last line uses the **conditional operator** `cond ? a : b` to turn that into 0.
- **`while` inside `for`.** It looks like $O(n^2)$, but it is not: `left` only ever moves forward, at most n times **in total** across the whole run. This is the "amortised" argument from Lesson 8: count the total moves of each pointer, not the moves per loop.
- **`min(best, right - left + 1)`.** `std::min` from `<algorithm>`. Both arguments must have the **same type**; `min(best, 5LL)` would not compile because one is `int` and one is `long long`. A window from `left` to `right` inclusive has `right - left + 1` days.
- **`vector<long long> prefix(n + 1, 0);`** The constructor with two arguments: size, then starting value. Size $n + 1$ because $P_0 = 0$ comes first (the odometer from Lesson 5 starts at zero).
- **`prefix[i + 1] = prefix[i] + vol[i];`** Here `vol[i]` is an `int` and `prefix[i]` is a `long long`. C++ converts the `int` up to `long long` before adding (the "usual arithmetic conversions" from Lesson 11), so there is no overflow.
- **`int lo = 1, hi = n;` and `while (lo < hi)`.** This is the "first green post" search with the answer inside the closed range $[lo, hi]$. We already checked that $L = n$ is green (the total reaches T), so the answer exists. When `mid` is green we keep it (`hi = mid`); when red we skip past it (`lo = mid + 1`). The loop stops when `lo == hi`, and that post is the first green one.
- **`lo + (hi - lo) / 2`.** The same safe midpoint as Lesson 10. For these sizes `(lo + hi) / 2` would not overflow, but the habit costs nothing.
- **Testing with `assert` and `mt19937`.** In the full file, `struct Case { ... }` groups each example's input and expected answer, and `const auto& c : cases` loops over them by reference. `mt19937 rng(20261108)` is a random generator with a fixed seed, so a failing test can be repeated exactly. `uniform_int_distribution<int>(1, 30)(rng)` builds a distribution and draws one number from it in a single line.

3.8 Self-check (answers below)

1. Why does the sliding window need all volumes to be positive? Give a 3-number array with a negative number where it gives the wrong answer.

2. In the binary search version, why is it safe to return `lo` at the end without checking it once more?
3. Which of this week's lessons would you use to check whether a string of brackets like `([)]` is balanced?

Answers. (1) With negatives, dropping a day from the back can **raise** the total, and a later day can lower it, so "shrink only while good" can miss the best window. Example: `[1, -5, 6]` with $T = 6$. The window reaches `[1, -5, 6]` with sum 2 (not good), never shrinks, and misses the single day `[6]`. It returns 0 instead of 1. (2) Because we checked before the loop that $L = n$ is green, so the first green post exists inside $[1, n]$, and the loop keeps it inside $[lo, hi]$ until only one post is left. (3) Stacks (L12): push each opening bracket, and on each closing bracket the top of the stack must be its partner.

---

4. Music: Derek and the Dominos, "Layla" (1970)

Context

"Layla" is the title track of **Layla and Other Assorted Love Songs**, a double album released in November 1970 by Derek and the Dominos. The "band" was really Eric Clapton in disguise: after Cream and Blind Faith he wanted to escape his own fame, so he hid inside a group with Bobby Whitlock (keyboards and vocals), Carl Radle (bass) and Jim Gordon (drums). The song is credited to Clapton and Gordon. It was recorded in Miami in the late summer of 1970 with the producer Tom Dowd, and Duane Allman of the Allman Brothers Band joined the sessions on slide guitar.

The story behind it

Clapton had fallen in love with Pattie Boyd, who was then married to his close friend George Harrison of the Beatles. He could not say it out loud, so he wrote an album about it. The name comes from an old Persian love story, "Layla and Majnun", told by the 12th-century poet Nizami, about a young man driven mad by a love he cannot have. Clapton saw himself in it.

At first the album sold poorly. Few people knew that "Derek" was Clapton, and the band broke up within a year. "Layla" only became a hit when it was released again as a single in 1972, reaching the top ten in both Britain and America. Boyd later left Harrison and married Clapton in 1979. In 1992 Clapton recorded a slow acoustic version for his "Unplugged" album, and it won the Grammy for Best Rock Song.

Why people love it

It is really two songs joined together. The first half is pure anguish: a fast, desperate guitar riff and a singer begging. The second half is a long, calm piano piece, as if the storm has passed and only the ache is left. That contrast is the whole emotional story in seven minutes. Many people also know the piano ending from the film "Goodfellas" (1990).

What to listen for

- **The opening riff.** Several guitars play it together, stacked on top of each other, which is why it sounds so huge. It is one of the most famous riffs in rock.
- **Two guitarists talking.** Clapton and Allman trade lines throughout. Clapton’s tone is thicker and bluesier; Allman’s slide is the sharper, singing voice that cries above it.
- **The change at the halfway point.** Listen for the moment the band drops away and the piano coda begins. Notice how the drums go from pounding to gentle.
- **The very end.** Allman’s slide plays high, bird-like notes over the piano. Listen to how quietly it all fades out, the opposite of how it started.